

AlphaStrategy: final prep brief

McGill-FIAM Asset Management Hackathon 2026 · Stage Two (Final Hack) · code state: branch feature/claudeCorrections, commit 0447ea3 · 10 Oct 2026

What this is. Every concept, model and decision behind our strategy on a few pages, then the weak points, the questions the jury is likely to ask, the improvement paths, and a checklist of what to verify in the code before the final. Numbers marked *earlier runs* come from PIPELINE.md and our logs, not from the final run. Take the final figures from `output/` (where to find each one is listed in section 12).

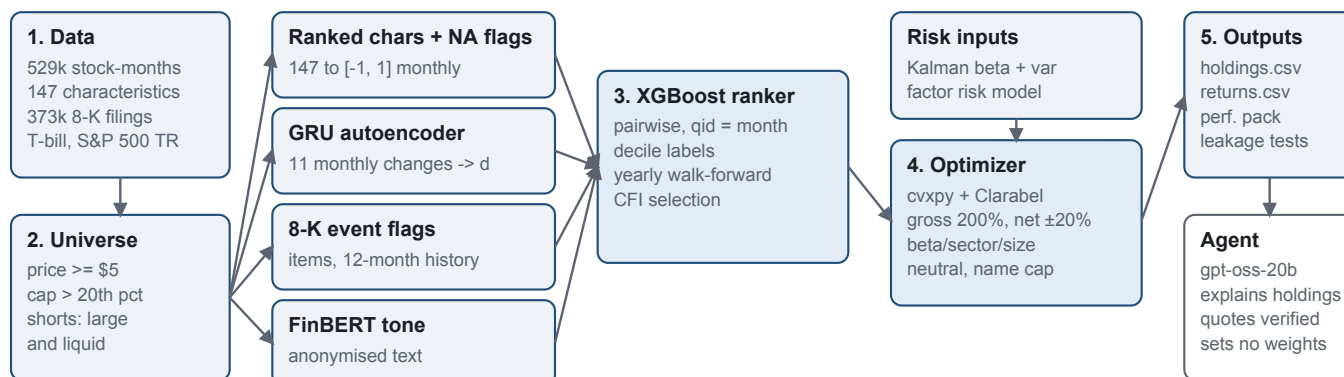
1. What the final judges

Rule or criterion	Competition requirement	Where we stand
Positions	100 to 500 names, both legs together, every month	Spec book: at least 120 by construction. v2/v3: no explicit floor, so check it (section 12)
Gross exposure	At most 200% of capital	Spec: exactly 200%. v2: at most 200% (it de-levered to about 127% in earlier runs)
Net exposure	Inside -50% / +50% at every rebalance	Our band is $\pm 20\%$ (v4: 0)
Benchmark	TB3MS/100/12 + 0.04/12 per month, a time series	metrics.book_returns: total = rf + excess, active = excess - 0.04/12
Headline metric	$IR = \sqrt{12} \cdot \text{mean}(\text{active}) / \text{sd}(\text{active})$	_pipe_ir, metrics._ir
Neutrality	Realised beta vs S&P 500, with its standard error. Checked before anything else	OLS with Newey-West (3 lags) in performance.json, plus rolling 12-month beta chart
Also read	Sharpe, max drawdown, alpha and t-stat, turnover, short-book tradability, top 10 long and short (ticker + name), OOS R^2	All in performance.json and figures/

- **Format of the final:** top 10 teams, two days, mentoring by working PMs, expanded 8-K coverage, and alternative data from Carbon Arc (card and receipt panels, web and app traffic, job postings, supply chain). Carbon Arc may award a separate prize for *originality* of use, judged apart from the main ranking.
- **What the brief rewards:** the reasoning behind each tool ("why this technology, for this signal, on this data"), honesty about failures, and an audit trail for anything an agent did. A thesis written by an agent is not evidence; the backtest is.
- **Two habits the organisers recommend:** (1) any new feature block must merge on a (permno, month) key without rewriting downstream code; (2) arrive knowing which of our signals is weakest and which question we want the new data to answer.

2. The strategy in one paragraph

At each month-end t we rank the stocks of a liquid US universe by expected next-month excess return. The ranker is an XGBoost model with a pairwise loss, retrained once a year in a walk-forward schedule. Its inputs are the 147 characteristics (ranked each month), GRU embeddings that summarise how those characteristics moved over the past year, 8-K event flags, and FinBERT tone. A convex optimizer turns the ranking into a long/short book with 200% gross that is neutral, ex ante, to beta, sector and size. The book is held during month $t+1$ and rebuilt every month. A local LLM agent then explains the largest holdings from the filings, with every quoted fact checked by code. The agent never changes the weights.



Conventions used everywhere. A row dated t uses month- t data only. The target is `ret_exc_lead1m` (excess return of month $t+1$), never shifted again. The holding date is the first day of $t+1$. Weights are signed fractions of \$1 of capital: $\text{excess} = \sum w_i r_i$, $\text{total} = \text{TB3MS}_{t+1}/1200 + \text{excess}$, $\text{active} = \text{excess} - 0.04/12$. Seed 42. Every setting sits in `CONFIG` (`src/config.py`, top of `MAIN.py`).

3. Data, universe and preprocessing

Step	What we do	Why
Universe (<code>data.py:36</code>)	Each month: $ \text{price} \geq \5 and market cap strictly above the 20th percentile of all stocks that month	Tradable names only. Nano-caps flatter backtests (the teammate's original book drew about 2/3 of its P&L; from nano and sub-\$5 names)
Short-eligible (<code>data.py:29</code>)	Market cap ≥ 40 th percentile (tuned over 30/40/50), dollar volume ≥ 30 th percentile, zero-trade days $\leq$ the month's median	Borrow cost and recall risk. The jury reads the short book: average market cap, dollar volume, small-cap share
Rank transform (<code>data.py:58</code>)	Each feature dense-ranked within the month, scaled to $[-1, 1]$, missing $\rightarrow 0$ (the median)	Comparable across features and months; robust to extreme raw values (raw betas reach $\pm 10^4$); same method as the template
Missing flags	A 0/1 column for every feature that is more than 20% missing on pre-2019 training rows	Keeps the fact that a value was imputed, which the rank transform would otherwise erase
8-K cleaning	Deduplicate on (permno, text hash), keep the earliest filing	The same text is not counted twice
Sector	First two digits of GICS ("NA" if missing)	Used for neutrality and for the Kalman prior

4. Inputs to the model and the optimizer

4.1 GRU autoencoder: how the characteristics are moving (`gru.py`)

Concept. A sequence autoencoder. A GRU encoder reads a stock's last 11 month-to-month changes in its 147 ranked features (an 11×147 matrix) and compresses them into d numbers. A GRU decoder must rebuild the sequence from those d numbers. Because the training target is the input itself, returns never enter: there is no label leakage. **Settings.** Hidden size 64, d in $\{8, 16, 32\}$, masked MSE loss, AdamW (lr $1e-3$, weight decay chosen from $\{0, 1e-4, 1e-3\}$), early stopping on 2019-2020 reconstruction loss, gradient clipping at 1.0. Trained on months up to 2018-11, then frozen. A window needs at least 6 valid changes, otherwise the embedding is NaN. **Why.** The ranked features are a snapshot. Whether a firm is improving or deteriorating is information a snapshot misses. **Limits.** Never retrained after 2018. Reconstruction quality is not return relevance, so ablation 3 must show it helps.

4.2 8-K event flags (`text.py`)

Each filing is assigned only to the calendar month of its `filing_date`. For each stock-month: `has_filing`, `n_filings`, and 0/1 flags for 14 items (1.01, 1.02, 1.03, 2.01, 2.02, 2.03, 2.05, 2.06, 3.01, 4.01, 4.02, 5.02, 7.01, 8.01). Plus 12-month trailing counts of the distress items 4.02 (non-reliance), 5.02 (officer changes), 2.06 (impairments), 1.02 (terminated agreements) and 2.05 (restructuring). **Timing:** a filing dated in month t is used to form the book at the end of t , which is held during $t+1$. That is conservative and matches the brief ("accumulate the news within month t ").

4.3 FinBERT tone (text.py)

Concept. FinBERT is a BERT model fine-tuned to classify financial text as positive, negative or neutral. Tone of a 512-token chunk = $P(\text{positive}) - P(\text{negative})$. Each filing is cut into at most 8 chunks, and we keep the mean and the minimum over chunks. Per stock-month: `tone_mean`, `tone_min`, and `tone_surprise` = the month's tone minus the firm's average over months $t-12$ to $t-1$. **Look-ahead control.** FinBERT's pretraining predates 2021, and the firm's name and ticker are removed before scoring, so the model cannot recall what happened to the firm later. Tickers are matched case-sensitively so that tickers that are common words (ALL, NOW, FOR) do not delete ordinary text. **Limits.** Only about the first 4,000 tokens are read; exhibits (often where the earnings numbers are) are missed; many 8-Ks are boilerplate.

4.4 Kalman-filtered beta: a risk input, not a signal (kalman.py)

Problem. The data carry four noisy betas per stock: Dimson 21-day (fast, very noisy), downside 252-day, BAB 1260-day (stable, slow) and 60-month. The optimizer needs one reliable beta per stock to enforce neutrality.

State: $\beta_t - m_t = \varphi(\beta_{t-1} - m_{t-1}) + \eta_t$, $\eta \sim N(0, q)$ · Observations: $y_{j,t} = \beta_t + \varepsilon_{j,t}$, $\varepsilon \sim N(0, R_j)$ · m_t = sector median of `beta_60m`
Each month: predict ($d \leftarrow \varphi d$, $P \leftarrow \varphi^2 P + q$), then update with each available measure using the gain $K = P/(P + R_j)$. Outputs: `beta_kf` = $m_t + d$ and `beta_var` = P . φ , q and the four R_j are estimated by maximum likelihood on months up to 2018-12, then frozen. The filter runs forward only (no smoother), so it is causal. Raw betas are clipped to the 1st-99th percentile first. **Why Kalman rather than one beta or an average:** (1) it weights each measure by its estimated precision; (2) it follows a beta that changes over time; (3) it shrinks extreme values toward the sector, which stops the optimizer from exploiting beta estimation errors; (4) it gives an uncertainty per stock, which the spec book penalises ($\lambda_\beta \cdot \sum \text{beta_var} \cdot w^2$). **Limits:** we use the filtered estimate at t , not the one-step prediction for $t+1$. If the likelihood pushes $R(\text{betabab}_{1260d})$ toward zero, `beta_kf` just copies the 5-year beta. Check `cache/kalman.json`.

Beta check (step 10). Regress the 2019-2020 validation book on the S&P 500. If $|t| > 1.96$, try four beta variants (`kf`, `kf_rfloor` with a floor on every R_j , `b60_shrunk` = $0.67 \cdot \text{beta}_{60m} + 0.33$, and `factor` = the risk-model beta) and keep the one whose validation book beta is closest to zero.

5. The model: XGBoost pairwise ranker (ranker.py)

Concept. Gradient-boosted trees trained with a *pairwise ranking* loss. Each month is one query group (`qid = eom`). For pairs of stocks in the same month, the model is penalised when it orders them differently from their realised returns. Labels are within-month deciles (0-9) of next-month excess return. **Why a ranker.** A market-neutral book only needs the order of stocks within a month, not the level of their returns. Decile labels are robust to return outliers. Trees capture nonlinearities and interactions between characteristics that Ridge cannot. **Fixed settings:** learning rate 0.03, subsample 0.7, colsample 0.5, `min_child_weight` 200, `reg_lambda` 10, `hist`.

Critical fix to be able to explain. In `xgboost 2.x` the defaults broke the model: pairs were built only among the top-ranked stocks (`pair_method="topk"`) and gradients were normalised so heavily that `min_child_weight=200` blocked every split. The first full run had a negative test IC. We restored the classic behaviour (`lamdarank_pair_method="mean"`, `lamdarank_normalization=False`): on synthetic data the IC went from 0.006 to 0.021.

Walk-forward schedule (all splits by *target* month, refitted yearly, monthly forecasts):

Test year	Train	Validation	Test
2021	2015-02 to 2018-12	2019-01 to 2020-12	2021-01 to 2021-12
2022	2015-02 to 2019-12	2020-01 to 2021-12	2022-01 to 2022-12
...	expanding	rolling 2 years	one year
2026	2015-02 to 2023-12	2024-01 to 2025-12	2026-01 to 2026-08

- **Tuning per window:** depth {3, 4, 5} × trees 50 to 2,000 (every 50) = 120 points, chosen by mean monthly rank IC on validation. Then refit on train + validation and predict the test year.
- **CFI feature selection (cfi.py).** Cluster the 147 factors on training months (median monthly Spearman, distance $1 - |\rho|$, complete linkage, cuts at $|\rho| = 0.65 / 0.75 / 0.85$). Measure each cluster's importance by permuting the whole group on validation (3 seeds). Keep the top 25 / 50 / 75% of groups and choose the subset with the best validation IC; keep all 147 if nothing beats them. *Why:* correlated factors split importance between them, so grouping gives a fairer measure. Teammate's result: test IC 0.065 → 0.088, with selections that change a lot from year to year.

- **OOS R² (required by the brief):** ret_hat is a linear map from score rank to return, fitted on validation, then $R^2 = 1 - \frac{\sum (r - \text{ret_hat})^2}{\sum r^2}$ (zero benchmark, as defined in the brief). It measures the calibration of that map.
- **Baseline:** Ridge on the 147 characteristics (the template), ablation 1.

6. Portfolio construction (optimizer.py, risk.py)

Factor risk model. Exposures = intercept + 9 ranked characteristics (size, book-to-market, momentum 12-1, reversal 1-0, beta_60m, idiosyncratic vol, profitability, asset growth, dollar volume) + sector dummies. Monthly cross-sectional OLS of realised returns gives factor returns. F = their covariance over the trailing 60 months (at least 24), shrunk 10% toward the average variance. Specific variance comes from residuals over 36 months. Factor betas to the S&P 500 give a model beta. Only returns realised by month t are used.

	Spec book (v1)	v2	v3 "classic"	v4 (teammate port)
Objective	$\max s \cdot w - \lambda_{tc} \Delta w - \lambda_{\beta} \sum \text{beta_var} \cdot w^2 - \lambda_{risk} \ w\ ^2 - \lambda_{fac}$ $w'XFX'w$	$\max \alpha \cdot w - 12 \text{ bps} \cdot \Delta w $, with $\alpha = IC \times$ cross-sectional vol $\times z$	No optimizer: top/bottom 10% within each sector, equal weight	$\max \text{rank-score} \cdot w - \text{risk}$ penalty (Ledoit-Wolf) – turnover penalty
Candidates	Top 250 / bottom 250 (short-eligible)	Same	Sector tails, shrunk to $\leq$ 500 names	Top 250 / bottom 250
Cap per name	1.5%	0.5%	1.5%	None (validator checks 100-500 names)
Gross / net	= 200% / $\pm 20\%$	$\leq 200\%$ / $\pm 20\%$	200% / $\pm 20\%$	{100, 150, 200}% / 0
Beta	$ \beta_{kf} \cdot w \leq 0.02$ (relaxable)	$ \text{model } \beta \cdot w \leq 0.02$	Leg sizes set for neutrality	$ \text{beta_60m} \cdot w \leq 0.05$
Other risk	Sector $\pm 10\%$, size ± 0.05	Ex-ante vol $\leq 8\%$, sector, size	Sector-neutral by construction	Size-group balance
Tuned on	1,440-point grid on 2019-2020	Nothing (stated assumptions)	2 smoothing values	72 configs per year on its validation

Infeasible months. Spec and v2 widen the sector and size bands first (steps of 0.05), then the beta tolerance (steps of 0.01), and v2 then raises the vol cap. If nothing solves, they hold last month's weights and log `fallback_hold`.

Book selection (`book: "auto"`, `pipeline.py:435`): among spec, v2, v3_s1.0, v3_s0.5 and v4, keep the highest 2019-2020 validation IR among books with $|\text{validation beta}| \leq 0.2$. Test numbers are logged for transparency only.

Decision to take before the final: lock one book. Choosing the best of five IRs on 24 months selects noise (the standard error of an annual IR over 2 years is about 0.7). The rule also favours the most-tuned books (spec, v4), uses IR *gross* of costs, and compares books whose constraints differ. Earlier runs: spec 0.76 vs v2 0.38 on validation, 0.16 vs -0.20 on test. Recommendation: the book picked by the pre-declared rule in the final run becomes the book of record, frozen, with an ex-ante rationale. Present the others as robustness checks. Do not switch books after looking at test results.

7. Validation discipline and leakage tests

Choice	Grid	Chosen on	Frozen?
GRU weight decay, epochs	{0, 1e-4, 1e-3}, patience 5	2019-2020 reconstruction loss	Yes
Kalman ϕ , q , R_j	Continuous MLE	Likelihood, months $\leq 2018-12$	Yes
XGB depth $\times$ trees, CFI subset	120 points + up to 9 subsets per window	That window's validation IC	Per window
d , λ_{tc} , λ_{β} , short ME pct, λ_{risk} , λ_{fac}	3·4·4·3·2·5 = 1,440 books	2019-2020 validation IR (10a: d; 10b: the rest, on the CFI scores)	Yes
Beta variant	kf, kf_floor, b60_shrunk, factor	Validation book beta closest to 0	Yes
Book	spec, v2, v3 $\times 2$, v4	Validation IR, $ \beta \leq 0.2$	Yes

- **Asserted leakage tests** (`leakage_tests.json`; the run stops if one fails): no filing dated after its month; train < validation < test; the latest training target month is before 1 January of the test year; no feature looks like the target (by name, or correlation > 0.999); (permno, eom) unique.

- **Shuffled-label run:** the whole pipeline with returns permuted within each month. Test IC and IR should be close to 0. It gives a noise baseline; the asserts above are the real leakage tests.
- **Ablations (ablations.json):** 1 Ridge on 147 → 2 XGB on 147 → 3 + NA flags + GRU → 4 + 8-K events → 5 + FinBERT tone → 6 + CFI. Each reports IC, ICIR and book IR/beta, and for 4-6 also the IC on stock-months that have a filing.
- **Optimism to disclose:** depth/trees, CFI subset, the 1,440-point grid and the book choice all use the same 2019-2020 months, so the validation IR is optimistic. v2, v3, v4 and the beta-check changes were designed after seeing earlier test results (PIPELINE.md §8 item 19). Every choice still uses validation data only.

8. The analyst agent (agent.py)

Element	Implementation
Model	gpt-oss-20b served locally by Ollama (OpenAI-compatible API), at most 4 tool-calling steps
Tools	<code>get_filings(permno, before_eom)</code> : the 5 latest 8-Ks dated $\leq$ formation date, anonymised, 6,000 characters <code>max · get_profile(permno, eom)</code> : a few characteristics, no name or ticker · <code>get_score(permno, eom)</code> : model score and percentile
Output	Strict JSON: thesis, facts (each with a verbatim quote and a document_id), risks, confidence
Controls	Arguments from the LLM are untrusted: the permno is forced to the held stock and dates are capped at the formation date. A code verifier keeps a fact only if its quote appears in a filing dated $\leq$ the formation date. The verification rate is reported
Role	Explains the top 10 long and top 10 short holdings. It does not feed the signal or the weights.

To say upfront. The LLM's pretraining postdates the 2021-2025 formation dates, so its explanations are post hoc (the output file says so). Anonymisation and the date caps reduce recall but cannot rule it out. That is exactly why it does not trade. The brief values an honest account: show the prompt, the tool definitions and the verification rate.

9. Decision log: what we chose and why

Decision	Rationale	Disclosure
Ranker (pairwise) rather than return regression	Only within-month order matters for a neutral book; robust to outliers	-
Rank features and fill NaN with the median	Comparable scales, outliers neutralised, template-compatible	Missing flags keep the information
Strict universe and stricter short list	Tradability; the jury reads the short book	Short ME percentile tuned (deviates from the spec)
Beta neutrality ex ante, net $\pm 20\%$ (not $\pm 50\%$)	The brief: "a dollar-neutral book long high beta is a disguised long"	Realised beta was negative in earlier runs
Sector $\pm 10\%$ and size ± 0.05 neutrality	Removes unintended bets; size added because shorts are structurally large	Size: added after the first run, chosen on 2019-2020
L2 and factor-risk penalties	The linear objective piled names at the cap; realised vol about 21%/yr	Added after the first full runs
Kalman beta + variance penalty	One stable beta; avoids exploiting beta estimation errors	Raw betas clipped (not in the spec)
FinBERT plus anonymisation	Pretraining before 2021; no recall of the firm	Case-sensitive tickers (deviates from the spec)
Missing returns = 0	No delisting returns in the readme	Count reported (n_missing_returns)
S&P 500 = total return series	That is the file we have	State it in the deck
Returns gross of costs	Competition default	Report net at 10 bps per unit of turnover too

10. Weak points and the questions to expect

Question	Answer to prepare
"Is the book really neutral?"	Yes ex ante: $ \beta \cdot w \leq 0.02$ on the Kalman beta, net $\pm 20\%$, sector and size bands. Ex post, earlier runs showed a negative beta (-0.18 , then -0.28), from a defensive tilt: long low-beta quality names, short high-beta junk. Show the rolling beta chart and the beta-fix procedure. Be ready to explain the final number and its standard error.
"Why XGBoost and not a neural network?"	Tabular data, about 2,000 stocks a month: trees are the strongest and most stable choice (Gu-Kelly-Xiu 2020). The deep learning goes where it helps: the GRU for time dynamics, FinBERT for text.
"How did you rule out look-ahead, including from the LLMs?"	Splits by target month, asserted tests, a causal Kalman filter, everything frozen at 2018, FinBERT pretrained before 2021 with anonymised text, the agent capped at the formation date and kept out of the weights.
"How many configurations did you try?"	Give the counts: 120 per window for XGB, 1,440 books, 5 book types. Show the shuffled-label run and the ablations. Say that validation IR is optimistic and that the book was frozen before the test.
"Turnover and costs?"	Earlier runs: about 220% per month on the spec book. Give the IR net of 10 bps (<code>chosen_book_net_of_fee</code>) and the λ_{tc} chosen.
"Is the short book tradable?"	Short list = cap ≥ 40 th percentile, liquid, few zero-trade days. Quote the short book's average market cap, dollar volume and small-cap share from <code>performance.json</code> .
"What actually drives performance?"	The IC steps across ablations, the factor groups CFI keeps each year, the long vs short leg, and the top 10 positive and negative contributors.

11. Improvement paths

When	Improvement	Expected effect
Before the final (hours)	Freeze one book; select on IR net of costs; one shared set of hard constraints for all books (names 100-500, cap, net)	A clean, defensible methodology
	Use the one-step prediction $m_{t+1} + \phi \cdot d_t$ for the Kalman beta; check the R_j in <code>kalman.json</code>	Beta closer to 0 over the holding month
	Fix the ticker/name lookup (backward only) and verify unlabeled names	A compliant deck and holdings file
During the final (2 days)	Generic loader for a feature block keyed on (permno, month), lagged to month t	Plug in Carbon Arc on day 1
	Carbon Arc: card and receipt panels $\rightarrow$ revenue nowcast vs consensus for consumer names; job postings and web/app traffic as fundamental momentum	A signal outside prices and accounting; originality prize
	Event-triage agent (spec step A): material yes/no, event type, severity 0-3, verified quotes $\rightarrow$ stock-month features	Turns the agent into an audited signal
	Expanded text: read exhibits (8-K 2.02 = earnings) and earnings-surprise features	Better tone and event signal
Later	Borrow-cost proxy in the objective; explicit turnover budget	Net performance and tradability
	Daily measurement (CRSP or Alpha Vantage): daily vol, drawdown, VaR	Differentiator the brief asks for
	Explicit neutrality to momentum and value factors; risk-model beta as the main beta	A more honest alpha
	Ensemble rankers (several seeds or models); retrain the GRU per window	A more stable signal

12. Pre-final checklist

A. Issues found in the code today (commit 0447ea3)

Item	Where
☐ PIPELINE.md says <code>beta_check_t</code> was lowered to 0; the config is 1.96 (comment: "0.0 tested, rejected"). Fix the doc.	PIPELINE.md (end); config.py:122

Item	Where
☐ PIPELINE.md §2.7b says the book of record is "spec"; the config is "auto". Write in the final chosen book.	PIPELINE.md §2.7b; config.py:126
☐ The MAIN.py header describes the spec optimizer (1.5% cap, factor penalty). It must match the book of record.	build_main.py HEADER
☐ Labels: <code>join_asof(strategy="nearest")</code> can take a ticker or name from a <i>later</i> 8-K. The brief forbids an unverified later name. Use "backward" and verify the rest.	report.py:45
☐ v2 and v3 have no explicit floor of 100 names (and v2 has no gross floor). Validate the final holdings.csv.	optimizer.py:155, :237
☐ A <code>fallback_hold</code> month carries last month's weights without re-checking the rules. Count these months in the log.	optimizer.py:72, :181
☐ Book selection uses IR gross of costs; only v4 is tuned net of costs.	pipeline.py:440
☐ MAIN.py is in sync with <code>src/</code> (rebuilt: no diff). Rebuild after every edit (<code>python build_main.py</code>).	build_main.py
☐ pytest is not installed on the Mac: run <code>python -m pytest tests</code> on the GX10.	tests/

B. Submission validity (on the final output)

Item	Where
☐ Every month: $100 \leq \text{names} \leq 500$, $\text{gross} \leq 200\%$, $ \text{net} \leq 50\%$.	output/holdings.csv
☐ 68 months from 2021-01 to 2026-08; Date = first day of the holding month; WEIGHT in % of NAV, positive for longs.	holdings.csv, returns.csv
☐ The files come from the FULL run, not a smoke run (smoke writes to the same output/).	output/
☐ No unlabeled names left (watch for the WARNING in the log); every name in the deck as "TICKER, Company Name".	run log
☐ <code>n_missing_returns</code> reported; the asserted leakage tests passed.	performance.json; leakage_tests.json
☐ Package: deck as PDF (8 pages + up to 10 of appendix), holdings.csv, returns.csv, MAIN.py without data, CVs.	-

C. Numbers for the deck: where to find them

File	Keys
performance.json	ir, sharpe, alpha_annual + alpha_t, beta + beta_t, max_drawdown, hit_rate, calendar_years, long_leg / short_leg, gross / net / turnover (avg, min, max), n_long / n_short, book.short_* (market cap, dollar volume, small-cap share), top / bottom contributors, oos_r2
settings_log.json	choice (d, λ , short_me_q), choice_10a, beta_check / beta_fix, book_selection (validation and test of each book), cfi (factors kept per year), xgb_infos (validation IC, depth / trees), chosen_book_net_of_fee
ablations.json	IC, ICIR, IR and beta per ablation; test_has_filing_rows
leakage_tests.json	Assert results; shuffled-label IC and IR
agent_rationales.json	verification_rate, verified vs dropped facts, theses of the top holdings
cache/kalman.json	ϕ , q, R_t per measure (check that $R(\text{betabab})$ is not about 0)
figures/	cumulative, underwater, rolling active return, rolling IR, rolling beta, histogram, contributors

D. Messages to lock with the team

- Our definition of neutrality, in one sentence: dollar (net $\pm 20\%$) + beta (ex ante $|\beta \cdot w| \leq 0.02$) + sector ($\pm 10\%$) + size.
- Returns are gross of costs; net at 10 bps is shown next to them. The S&P 500 series is total return.
- The book of record and why it was chosen before the test; the others are robustness checks.
- One disclosure sentence on the changes made after seeing earlier results (v2, v3, v4, beta trigger).
- The weakest signal (per the ablations) and the specific question we will put to the Carbon Arc data.